UNIVERSIDAD DEL BÍO-BÍO

12 Análisis de manchas

Tutorial Procesamiento de Imagen con webcam

Estudiantes Practicantes:
Javiera Gutiérrez
Javiera Henríquez
Teresa Vidal

Profesor:
Luis Vera

Laboratorio CIMUBB

2023-1

Análisis de manchas

Para este proyecto, tomaremos nuestra captura a color y dependiendo de su Umbralización, analizaremos cuantas manchas hay y que porcentaje de la imagen ocupa.

Creamos otro archivo .py y añadimos las siguientes librerías, creamos la ventana con sus parámetros correspondiente y agregamos las funciones de inicio de cámara que aparecieron anteriormente en el tutorial.

12 Análisis de manchas.py

12 Análisis de manchas.py > ...

```python
import tkinter as tk
from tkinter import *
from PIL import Image
from PIL import ImageTk
import imutils
import cv2

# Crea ventana, define tamaño y título
ventana = tk.Tk()
ventana.geometry("1320x470")
ventana.resizable(0,0)
ventana.title("Umbralización RGB")

#Funciones cámara web
def camara():
    global capture
    capture = cv2.VideoCapture(0)
    iniciar()

def iniciar():
    global capture
    if capture is not None:
        ret, frame = capture.read()
        if ret == True:
            frame = imutils.resize(frame, width=311)
            frame = imutils.resize(frame, height=241)
            ImagenCamara = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            im = Image.fromarray(ImagenCamara)
            img = ImageTk.PhotoImage(image= im)
            LImagen.configure(image= img)
            LImagen.image = img
            LImagen.after(1,iniciar)
        else:
            LImagen.image = ""
            capture.release()
```

También añadimos la función **Capturar()** para tomar la foto y **rgb()** para umbralizar una captura a color.

```
#Función para tomar una foto
def Capturar():
    global Captura
    camara = capture
    return_value, image = camara.read()
    frame = imutils.resize(image, width=301)
    frame = imutils.resize(frame, height=221)
    Captura = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    im = Image.fromarray(Captura)
    img = ImageTk.PhotoImage(image= im)
    LImagenROI.configure(image= img)
    LImagenROI.image = img

def rgb():
    global img_mask, img_aux, bin_imagen
    Minimos = (int(SRedI.get()),int(SGreenI.get()),int(SBlueI.get()))
    maximos = (int(SRedD.get()),int(SGreenD.get()),int(SBlueD.get()))
    img_mask = cv2.inRange(Captura, Minimos, maximos)
    img_aux = img_mask
    img_mask = Image.fromarray(img_mask)
    img_mask = ImageTk.PhotoImage(image= img_mask)
    ImagenUmbra.configure(image=img_mask)
    ImagenUmbra.image = img_mask
    _, bin_imagen = cv2.threshold(img_aux, 0, 255, cv2.THRESH_BINARY_INV)
```

Creamos una nueva función llamada **manchas()** en la utilizaremos la imagen auxiliar, la máscara y la **bin_imagen** que se crearon en la función anterior, para esto, es importante que en **rgb()** queden definidas estas tres variables como **global**.

Contamos el número de píxeles con manchas y con esto calculamos el porcentaje de manchas en el total de la imagen.

Luego en una caja de texto le pasamos esta información para que la muestre en pantalla.

```
def manchas():
    # Contar el número de píxeles con manchas
    num_pixels_con_manchas = cv2.countNonZero(bin_imagen)
    # Calcular el porcentaje de manchas
    porcentaje_manchas = 100 - (num_pixels_con_manchas / bin_imagen.size) * 100
    #Contornos
    contornos = cv2.findContours(img_aux, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[0]
    #Cantidad de contornos
    num_formas = len(contornos)
    Cadena = f"Cantidad de manchas blancas: {num_formas}\nPorcentaje área con manchas: {round(porcentaje_manchas,2)}%"
    CajaTexto.configure(state='normal')
    CajaTexto.delete(1.0, tk.END)
    CajaTexto.insert(1.0, Cadena)
    CajaTexto.configure(state='disabled')
```

Se añaden las barras para que el usuario arrastre y seleccione los números RGB para calcular la Umbralización de la imagen.

```
#RGB se inicia en 1, ya que si no sale error de división por 0
SRedI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedI.place(x=760,y=300)
SGreenI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenI.place(x=760, y= 340)
SBlueI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueI.place(x=760, y= 380)

SRedD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedD.set(255)
SRedD.place(x=900,y=300)
SGreenD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenD.set(255)
SGreenD.place(x=900, y= 340)
SBlueD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueD.set(255)
SBlueD.place(x=900, y= 380)
```

Añadimos los **Label** y cuatro botones que accionarán las funciones de cámara web, tomar foto, umbralizar y analizar las manchas de la imagen.

```
#Label
LRed = tk.Label(ventana, text="R")
LRed.place(x=870,y=320,width=21,height=16)
LGreen = tk.Label(ventana, text="G")
LGreen.place(x=870,y=360,width=21,height=16)
LBlue = tk.Label(ventana, text="B")
LBlue.place(x=870,y=400,width=21,height=16)

#Botones
BCamara = tk.Button(ventana, text="Iniciar cámara", command=camara)
BCamara.place(x=150,y=330,width=90,height=23)
BCapturar = tk.Button(ventana, text="Tomar foto", command=Capturar)
BCapturar.place(x=500,y=330,width=91,height=23)
Umbra = tk.Button(ventana, text="Umbralizacion", command=rgb)
Umbra.place(x=840,y=430,width=80,height=23)
Manchas = tk.Button(ventana, text="Análisis de manchas", command=manchas)
Manchas.place(x=1120,y=330,width=120,height=23)
```

Añadimos tres cuadros de imagen grises para mostrar la cámara web, la captura de imagen y la imagen umbralizada por pantalla.

Además, se añade una caja de texto donde se desplegará la información del análisis de manchas.

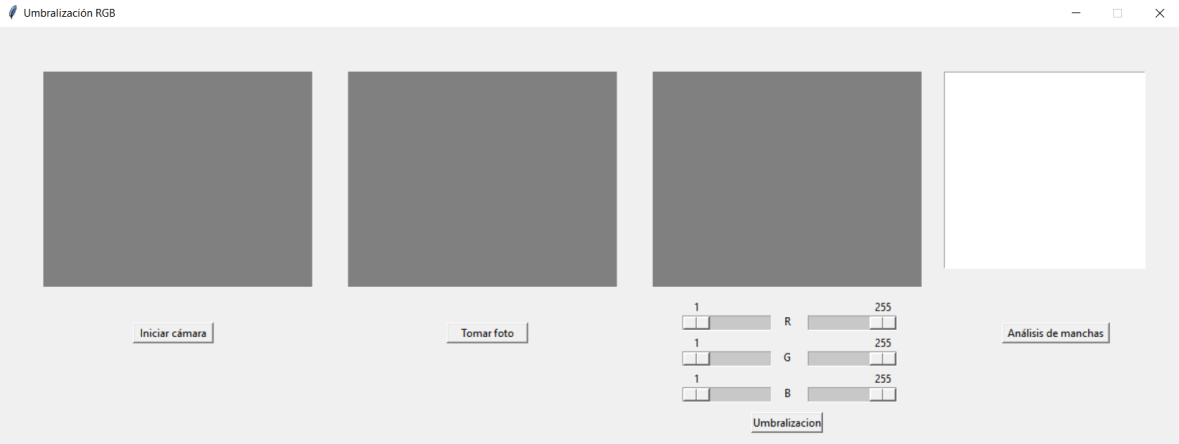
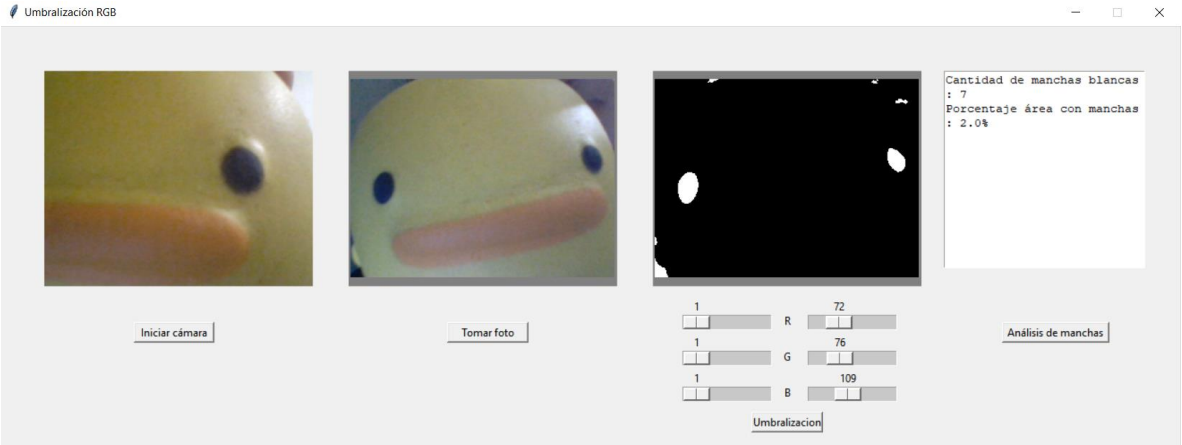
Finalmente, se añade el código **ventana.mainloop()** para terminar el programa y asegurarnos que todo se muestre correctamente.

```
#Cuadros de imagen gris
LImagen = tk.Label(ventana, background="gray")
LImagen.place(x=50,y=50,width=300,height=240)
LImagenROI = tk.Label(ventana, background="gray")
LImagenROI.place(x=390,y=50,width=300,height=240)
ImagenUmbra = tk.Label(ventana, background="gray")
ImagenUmbra.place(x=730,y=50,width=300,height=240)

#Cuadro de Texto
CajaTexto = tk.Text(ventana, state="disabled")
CajaTexto.place(x=1055,y=50,width=225,height=220)

ventana.mainloop()
```

Al compilar y ejecutar el programa se verá así:



Y al analizar una imagen se verá así: